

Assignment 3 VLSI

Siddarth Gottumukkula

October 2024

The Verilog Scripts, Test benches and the VCD files for all the questions from Q1 to Q11 are uploaded in the respective folders. An explanation is provided below for questions 9, 10 and 11. In all the questions, active low asynchronous reset has been implemented unless stated otherwise. For sequential circuits, the clock pulse has been shifted by 1 unit and given to the circuit. This is to ensure stable input levels before the clock edge, eliminating any undefined behaviour.

9 Question 9

9.1 Sequential circuit from the logical function

The given logical function is

$$\begin{aligned}A(t+1) &= xy' + xB \\ B(t+1) &= xA + xB' \\ z &= A\end{aligned}$$

This is implemented using 2 D flip flops, whose design was done in a earlier question. The above equations form the basis of the structural implementation of the design. However, I have implemented the circuit using the data flow modelling method. The truth table is given below.

$A(t)$	$B(t)$	X	Y	$A(t+1)$	$B(t+1)$	Z
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	1	1	0
0	0	1	1	0	1	0
0	1	0	0	0	0	0
0	1	0	1	0	0	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	1	1	1
1	0	1	1	0	1	1
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	1	1	1
1	1	1	1	1	1	1

Table 1: State Transition Table

The state diagram for this is drawn below.

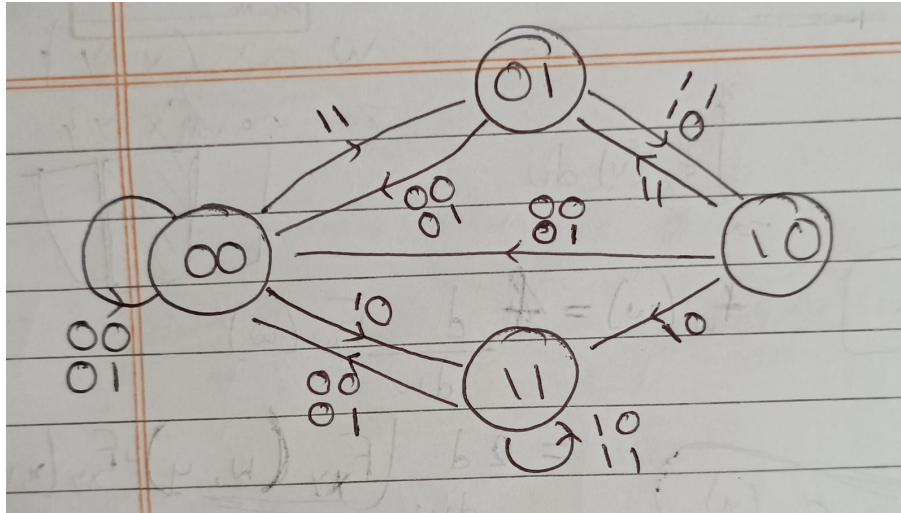


Figure 1: State Diagram

The inputs to the flip flops are governed by the current state, which is essential in sequential logic. However, for the purpose of testing, they have been declared as inputs. This makes it easier to test all the states and transitions from the test bench and ensure proper working of our design.

```

1  `timescale 1ns / 1ps
2
3  `include "DFF.v"
4
5  // For the sake of testing, we will also set the states of A and B
6  module Cir (input x, input y, input Clk, input Rst, output z, input TestA, input Test
7  B);
8  wire Aout, Not_Aout, Bout, Not_Bout;
9  wire Ain, Bin;
10
11  assign Ain = (x & (~y)) | (x & TestB); // Change Test to in
12  assign Bin = (x & TestA) | (x & ~TestB); // Change Test to in
13  assign z = Ain;
14
15  DFF A(.D(Ain), .Clk(Clk), .Reset(Rst), .Q(Aout), .Not_Q(Not_Aout));
16  DFF B(.D(Bin), .Clk(Clk), .Reset(Rst), .Q(Bout), .Not_Q(Not_Bout));
17
18  endmodule
  
```

Figure 2: Verilog Script

If we test for all the inputs from each of the states and ensure that the transition is right, the overall flow of the circuit will be right. Thus, I have written a test bench covering these cases. This is the reason we considered the inputs to the flip flops to be given by the test bench for the sake of testing. Had we not done that, the output would always cycle through certain states and not cover all the states as this design has internal loops in the state diagram. The test bench is provided below.

The clock pulse has been shifted by 1 unit to ensure that the inputs to the flip flops are stable at the positive clock edge.

```

1 'timescale 1ns / 1ps
2
3 module qa_Testbench();
4
5 reg x, y, A, B, Clk, Clr;
6 wire z, Aout, Bout;
7
8 Clr uut(.x(x), .y(y), .Clk(Clk), .Rst(Clr), .z(z), .TestA(A), .TestB(B), .Aout(Aout), .Bout(Bout));
9
10 initial
11 begin
12   Clk = 0;
13   Clr = 0;
14   #1
15   Clr = 1;
16   forever #5 Clk = ~Clr;
17 end
18
19 initial
20 begin
21   forever #10 $monitor("A = %b, B = %b, x = %b, y = %b, z = %b", A, B, x, y, z);
22 end
23
24 initial
25 begin
26   $dumpfile("qa.vcd");
27   $dumpvars(0, qa_Testbench);
28
29   A = 0;
30   B = 0;
31   x = 0;
32   y = 0;
33
34   for(integer i = 0; i < 16; i++)
35   begin
36     (A, B, x, y) = 1;
37     #10;
38   end
39   $finish;
40 end
41
42 end
43
44 endmodule

```

Figure 3: Test Bench

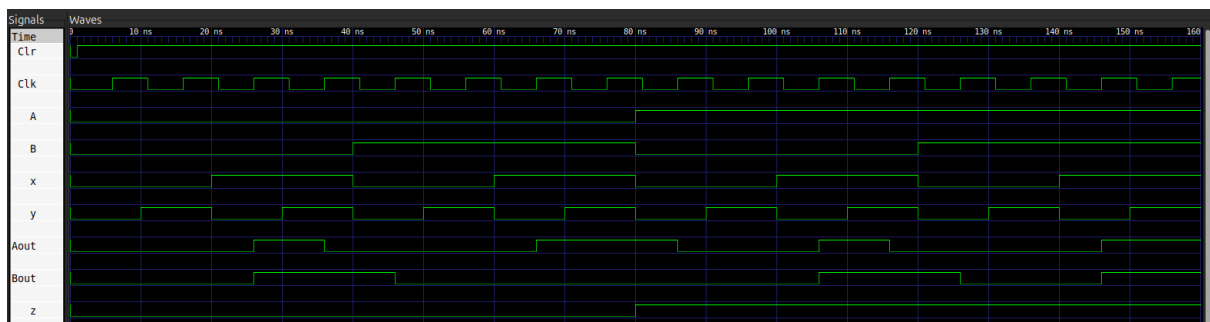


Figure 4: Plots

In the above plot, A and B are the current states and Aout and Bout are the next states. We can verify the working of the circuit from these plots as well! The output has also been printed in the terminal for reference.

9.2 Sequential circuit from circuit

The circuit which has to be implemented is given below. We have derived the state table from the circuit and from the state diagram, we can derive the state transition chart. All the these are given below in order.

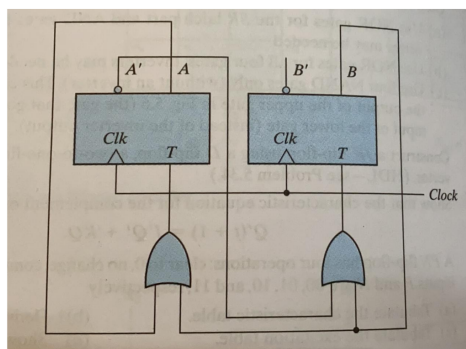


Figure 5: Circuit



```

1  `timescale 1ns / 1ps
2
3  module qb_Testbench;
4
5  reg Clk, Clr;
6  wire A, B;
7
8  Cir9b uut(.Clk(Clk), .Rst(Clr), .Aout(A), .Bout
9  (B));
10 initial
11 begin
12   Clk = 0;
13   Clr = 0;
14   #1
15   Clr = 1;
16   forever #5 Clk = ~Clk;
17 end
18
19 initial
20 begin
21   forever #10 $monitor("A = %b, B = %b", A, B);
22 end
23
24 initial
25 begin
26   $dumpfile("qb.vcd");
27   $dumpvars(0, qb_Testbench);
28
29   #100 $finish;
30 end
31
32 endmodule
33
34

```

Figure 9: Test Bench

Thus, as seen from the plots, the output is cyclic which matches our state table and diagram.

10 Question 10

Finite State Machines(FSMs) are of two types, Mealy and Moore. In the subsections, we implement the same logic in both the types and compare the results.

10.1 Moore FSM

The output is dependent only on the current state, not the input. I have modelled the FSM at a behavioural level. The *case()* statements come in handy while implementing the state transition logic, as seen in the script.

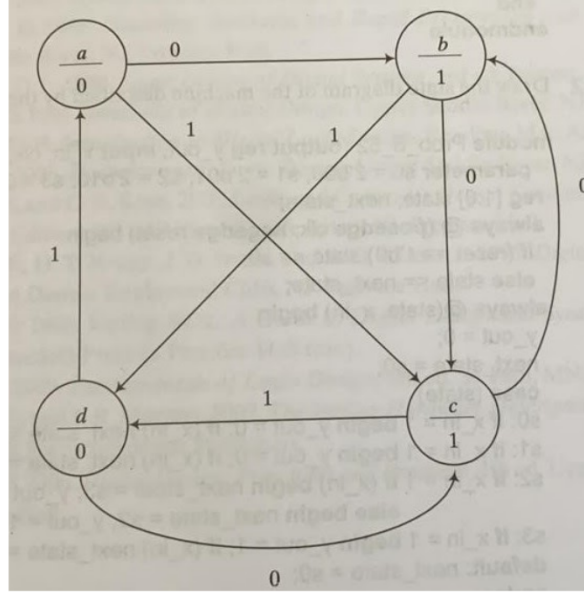


Figure 10: FSM to be implemented

Asynchronous active low reset has been implemented. On the positive edge of the clock, the current state gets updated. The next state of the machine is maintained in another register. This is done to keep the code more readable. With a slight modification, the need for this variable can be eliminated and the current state can be updated directly at the clock edge. As we are maintaining another register for the next state, there is another *always()* block which updates this register when any of the variables change. This is necessary as the next state register need not, in fact should not, be updated periodically (as the input can be changed between the clock cycles). When the FSM is reset, the current state is set to 0 and the next state is set based on the current input.

The state table is given below. We are considering the state A to be 00, B to be 01, C to be 10, and D to be 11, and we will call our flip flops P and Q which represent these states. An observation from the table is that the output Y is the XOR operation of the individual flip flop outputs, i.e., $Y = P \oplus Q$

P	Q	X	P(t + 1)	Q(t + 1)	Y
0	0	0	0	1	0
0	0	1	1	0	0
0	1	0	1	0	1
0	1	1	1	1	1
1	0	0	0	1	1
1	0	1	1	1	1
1	1	0	1	0	0
1	1	1	0	0	0

Table 3: State Table

```

1 // Output doesn't depend on input
2 `timescale 1ns / 1ps
3
4 module MooreFSM (input x, input Clk, input Clr, output reg
5 y);
6 reg [1:0] CurrState;
7 reg [1:0] NextState;
8
9 always @(posedge Clk or negedge Clr)
10 begin
11   if(!Clr)
12     begin
13       CurrState = 2'b00;
14       NextState = ((x > 0) ? 2'b10 : 2'b01);
15       y = 1'b0;
16     end
17   else
18     begin
19       CurrState = NextState;
20       y = CurrState[0] ^ CurrState[1];
21     end
22 end
23
24 always @(*)
25 begin
26   case(CurrState)
27     2'b00 : NextState <= ((x > 0) ? 2'b10 : 2'b01);
28     2'b01 : NextState <= ((x > 0) ? 2'b11 : 2'b10);
29     2'b10 : NextState <= ((x > 0) ? 2'b11 : 2'b01);
30     2'b11 : NextState <= ((x > 0) ? 2'b00 : 2'b10);
31     default : NextState <= 2'b00;
32   endcase
33 end
34
35 endmodule

```

Figure 11: Verilog Script for Moore

The test bench for the circuit is provided. Since the transition between states is not systematic, a certain input pattern is tested for covering all the states. In the comments of the test bench, the expected output is also given.

```

1 `timescale 1ns / 1ps
2
3 module MooreFSM_Testbench;
4
5   reg Clk, Clr, x;
6   wire y;
7
8   MooreFSM uut(.Clk(Clk), .Clr(Clr), .x(x), .y
9   (y));
10  initial
11  begin
12    Clk = 0;
13    Clr = 0;
14    #1
15    Clr = 1;
16    forever #5 Clk = ~Clk; // 6, 16, 26, ...
17  end
18
19  initial
20  begin
21    $monitor("Time=%0t, x=%b, y=%b", $time, x, y);
22  end
23
24  initial
25  begin
26    $dumpfile("MooreFSM.vcd");
27    $dumpvars(1, uut);
28
29    x = 0;
30    #30 // 0 -> 1 -> 2 -> 1
31    x = 1;
32    #40 // 1 -> 3 -> 0 -> 2 -> 3
33    x = 0;
34    #10 // 3 -> 2
35
36    $finish;
37  end
38 end
39
40 endmodule

```

Figure 12: Test Bench with the expected output in the comments

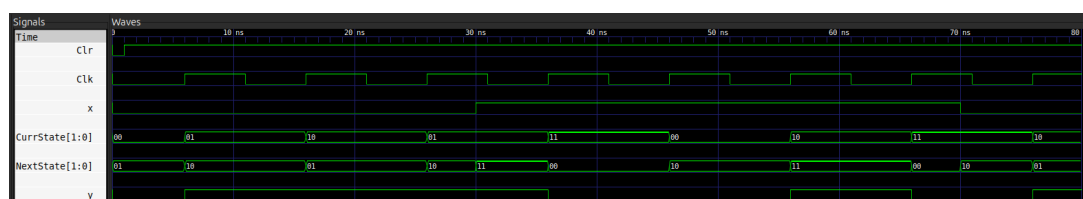


Figure 13: Result Obtained

On comparison, we find that our design is working just fine!

10.2 Mealy FSM

In the Mealy FSM, the output is dependent on the current state and the current input being supplied. Notice the slight change in the way the state diagram is drawn between the two. The output change need not be synchronous as the change in input in between the clock cycle might induce a change in the output state.

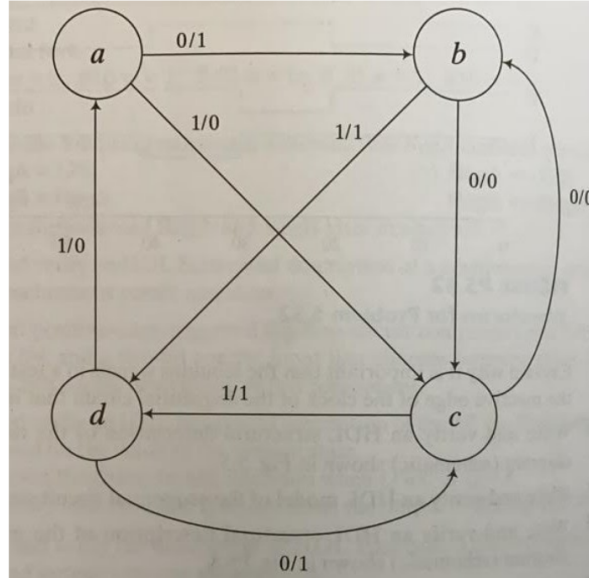


Figure 14: State Diagram

The change in the script between both these FSM types is that the output change is removed from the synchronous block and added to the asynchronous block. In essence, the output is changed whenever the input is changed and not only at the clock edges. Keeping this in mind The truth table of the circuit is given below, from which the logic for the output expression seen above is derived using K-maps.

We will follow a similar convention as elaborated above in the Moore FSM implementation.

P	Q	X	P(t + 1)	Q(t + 1)	Y
0	0	0	0	1	1
0	0	1	1	0	1
0	1	0	1	0	0
0	1	1	1	1	1
1	0	0	0	1	0
1	0	1	1	1	1
1	1	0	1	0	1
1	1	1	0	0	0

Table 4: State Table

The logical function for the output can be derived using K-maps and it evaluates to be the following expression.

$$Y = X(Q' + P'Q) + X'(P'Q' + PQ)$$

This is the expression used in updating the output in the verilog script.


```

1 // Output Depends on Input
2 `timescale 1ns / 1ps
3
4 module MealyFSM (input x, input Clk, input Clr, output reg y);
5
6 reg [1:0]CurrState;
7 reg [1:0]NextState;
8
9 always @(posedge Clk or negedge Clr)
10 begin
11   if(!Clr)
12   begin
13     CurrState = 2'b00;
14     NextState = ((x > 0) ? 2'b10 : 2'b01);
15     y = 1'b0;
16   end
17   else
18     CurrState = NextState;
19   end
20
21 always @(*)
22 begin
23   y = (~x & ~(CurrState[0] ^ CurrState[1])) + (x & (~CurrState[0] + (CurrState[0] & ~CurrState[1]))); // Y = X(B' + A'B) + X'(A'B' + A
24   case(CurrState)
25     2'b00 : NextState <= ((x > 0) ? 2'b10 : 2'b01);
26     2'b01 : NextState <= ((x > 0) ? 2'b11 : 2'b10);
27     2'b10 : NextState <= ((x > 0) ? 2'b11 : 2'b01);
28     2'b11 : NextState <= ((x > 0) ? 2'b00 : 2'b10);
29     default : NextState <= 2'b00;
30   endcase
31 end
32
33 endmodule

```

Figure 15: Verilog Script

From the truth table, we understand that Mealy machine and Moore machine are NOT two different ways of implementing the same FSM. They serve different purposes as the output change is different in both of them. The test bench and the plots for the same are provided below.

```

1 `timescale 1ns / 1ps
2
3 module MealyFSM_Testbench;
4
5   reg Clk, Clr, x;
6   wire y;
7
8   MealyFSM uut(.Clk(Clk), .Clr(Clr), .x(x), .y
9   (y));
10  initial
11  begin
12    Clk = 0;
13    Clr = 0;
14    #1
15    Clr = 1;
16    forever #5 Clk = ~Clk; // 6, 16, 26, ...
17  end
18
19  initial
20  begin
21    $monitor("Time=%0t, x=%b, y=%b", $time, x, y);
22  end
23
24  initial
25  begin
26    $dumpfile("MealyFSM.vcd");
27    $dumpvars(1, uut);
28
29    x = 0;
30    #30 // 0 -> 1 -> 2 -> 1
31    x = 1;
32    #40; // 1 -> 3 -> 0 -> 2 -> 3
33    x = 0;
34    #10 // 3 -> 2
35
36    $finish;
37  end
38
39
40 endmodule

```

Figure 16: Test Bench

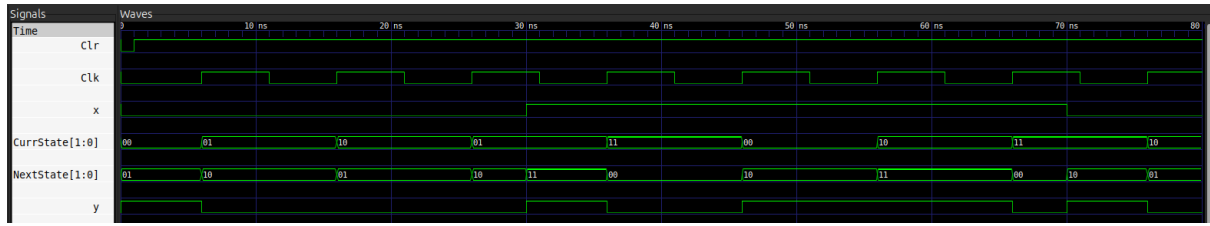


Figure 17: Result

We observe that for the same test bench, Mealy and Moore FSMs give us different outputs. Thus, depending on the requirement, one of the models can be chosen.

The Mealy FSM is always synchronous, even the output. This is because the output depends only on the current state and the current state changes only on the clock edges. Whereas, a Moore FSM may not be synchronous. This behaviour is induced by the dependence on the inputs, which may change in between the clock cycle as well.

Nevertheless, the state transition remains the same in both the cases. Only the output would be different owing to the dependence on the inputs in a Mealy FSM.